Домашнее задание

Устойчивость алгоритма
и корректная постановка эксперимента

Формат сдачи: файл `.py` (или `.ipynb`) с решениями и короткие выводы по каждой задаче (2–3 предложения). Для экспериментальных задач обязательны таблица или график.

Блок 1. Обусловленность задачи (задачи 1–4)

Задача 1. Число обусловленности вручную.

Для каждой функции вычислите число обусловленности cond = |x · f'(x) / f(x)| в указанной точке и определите, хорошо или плохо обусловлена задача:

1. f(x) = √x при x = 10^{-8}
2. f(x) = ln x при x = 1.001
3. f(x) = x − 1 при x = 1.000001
4. f(x) = sin x при x = π/2

Вывод: расположите задачи по возрастанию обусловленности. Какая из них самая «опасная» и почему?

Задача 2. Вычитание близких чисел как плохая обусловленность.

Дана задача f(a, b) = a − b. Покажите аналитически, что при a ≈ b число обусловленности по a и по b велико. Затем проверьте численно: возьмите a = 1 + 10^{-8}, b = 1 и посчитайте Δf / δa при δa = 10^{-9}. Совпадает ли фактическое изменение результата с предсказанием по cond? Объясните в 2–3 предложениях.

Задача 3. Кратные корни — плохая обусловленность.

Рассмотрите уравнение $x^2 - 2x + 1 = 0$ (корень x = 1 кратности 2). Измените свободный член на 1 − ε для ε = 10^{-2}, 10^{-4}, 10^{-6}. Найдите корни через `math.sqrt` и посмотрите, насколько сдвинулся корень относительно x = 1. Постройте таблицу: ε → сдвиг корня → отношение сдвигов. Сделайте вывод о связи кратности корня и обусловленности.

Задача 4. Обусловленность vs устойчивость — различие.

Приведите **свой** пример (не из лекции) задачи, которая **хорошо обусловлена**, но для которой можно написать **неустойчивый** алгоритм. Реализуйте оба: устойчивый и неустойчивый варианты, покажите расхождение на диапазоне входных данных. В выводе чётко разделите: «в чём проблема задачи» и «в чём проблема алгоритма».

Блок 2. Устойчивость алгоритма (задачи 5–9)

Задача 5. Квадратное уравнение — классика.

Для a = 1, b = 10^8, c = 1 вычислите меньший корень двумя способами:

- «плохо»: $x_2 = (-b + \sqrt{(b^2 - 4ac)}) / 2a$
- «хорошо»: $x_2 = 2c / (-b - \sqrt{(b^2 - 4ac)})$

Сравните с эталоном через `mpmath` (или `decimal` с 50 знаками). Посчитайте относительную погрешность обоих способов. Повторите для b = 10^4 и b = 10^{12}. Постройте таблицу: b → ошибка плохого → ошибка хорошего. Сделайте вывод: при каком b «плохой» способ начинает катастрофически ошибаться?

Задача 6. (1 − cos x) при малых x.

Реализуйте два способа вычисления $f(x) = (1 - \cos x) / x^2$:

- «плохо»: прямо в лоб;
- «хорошо»: через тождество $2 \cdot \sin^2(x/2) / x^2$.

Прогоните x = 10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}. Эталон — через `mpmath`. Постройте таблицу относительной ошибки обоих способов от x в log-log масштабе. Определите, при каком x ошибка «плохого» способа превышает 1%. Сформулируйте правило, когда нужно искать тригонометрическое тождество.

Задача 7. √(x+1) − √x при больших x.

Реализуйте два способа:

- «плохо»: $\sqrt{(x+1)} - \sqrt{x}$;
- «хорошо»: $1 / (\sqrt{(x+1)} + \sqrt{x})$.

Прогоните x = 10^4, 10^8, 10^{12}, 10^{14}, 10^{16}. Эталон — `mpmath` с 50 знаками. Постройте таблицу относительной ошибки. Определите, как ошибка «плохого» способа растёт с x (оцените показатель степени). Сделайте вывод.

Задача 8. Экспонента и логарифм.

Для $f(x) = \ln(1 + x)$ при малых x реализуйте:

- «плохо»: `math.log(1 + x)`;
- «хорошо»: `math.log1p(x)`.

Прогоните x = 10^{-2}, 10^{-8}, 10^{-12}, 10^{-16}. Эталон — `mpmath.log(1 + x)`. Посчитайте относительную ошибку обоих способов. Объясните, что делает `log1p` внутри (загляните в документацию Python) и почему он устойчив. Сделайте вывод о том, зачем в стандартной библиотеке есть «странные» функции типа `log1p`, `expm1`, `hypot`.

Задача 9. Неустойчивость накопления в сумме.

Вычислите $S_n = \Sigma\ (k = 1..n)\ 1/k$ для $n = 10^5, 10^6, 10^7$ тремя способами:

1. наивный цикл слева направо;
2. `sum(arr)`;
3. `math.fsum(arr)`.

Эталон — `mpmath` с 50 знаками. Для каждого n посчитайте относительную ошибку всех трёх способов. Постройте таблицу ошибки от n в log-log. Оцените, как растёт ошибка наивного способа с n. Сделайте вывод: при каком n наивная сумма становится неприемлемой (ошибка > 10^{-10})?

Блок 3. Корректная постановка эксперимента (задачи 10–15)

Задача 10. Разделение источников ошибки.

Возьмите задачу из задачи 6. Проведите **три** эксперимента:

- (а) меняем x, фиксируем алгоритм (плохой) — видим рост ошибки из-за обусловленности/сокращения;
- (б) фиксируем x, меняем тип float (float32 через `numpy.float32` vs float64) — видим рост ошибки из-за машинной точности;
- (в) меняем алгоритм (плохой vs хороший), фиксируем x — видим эффект устойчивости.

Оформите таблицу: какой фактор на сколько порядков меняет ошибку. Сформулируйте вывод: какой источник в каждом случае доминирует.

Задача 11. Эталон — обязателен.

Придумайте (или возьмите из задач 5–9) пример, в котором два **float-результата** совпадают между собой, но **оба неверны** относительно точного эталона. Реализуйте: два алгоритма, сравнение друг с другом через `==`, сравнение каждого с `mpmath`. Сделайте вывод о том, почему сравнивать float с float без эталона — ошибка.

Задача 12. Диапазон, а не одна точка.

Для функции из задачи 7 прогоните x от 10^2 до 10^{16} (минимум 8 точек, логарифмическая сетка). Постройте график ошибки в log-log. Оцените наклон прямой (показатель степени). Сравните с теоретической оценкой: ошибка «плохого» способа $\approx \varepsilon_{machine} \cdot x$. Совпадает ли наклон с предсказанием? Сформулируйте вывод о том, зачем нужен диапазон, а не одна точка.

Задача 13. Воспроизводимость эксперимента.

Напишите эксперимент, который генерирует 10^5 случайных чисел через `random.seed(42)` и `numpy.random.default_rng(42)`. Суммируйте их тремя способами (`sum`, `math.fsum`, `numpy.sum`). Запустите **дважды** с одним seed — результаты должны совпасть. Запустите с `seed=43` — покажите, что расхождения между способами не «случайны», а систематичны. Оформите отчёт с указанием: версия Python, версия numpy, seed. Сделайте вывод о том, почему без фиксации seed и версий эксперимент не воспроизводим.

Задача 14. Границы представимости.

Проверьте поведение наивного суммирования и `fsum` на массиве, содержащем:

- очень большие числа ($\approx 10^{300}$),
- очень маленькие ($\approx 10^{-300}$),
- ноль,
- денормализованные числа ($\approx 10^{-320}$),
- `float('inf')`,
- `float('nan')`.

Что произойдёт с каждым способом? Какой способ «отравится» NaN сразу, а какой — нет? Как ведёт себя `fsum` при наличии `inf`? Оформите таблицу поведения и сделайте вывод: как обрабатывать граничные значения в реальном коде.

Задача 15. Полный корректный эксперимент.

Выберите **одну** функцию из задач 5–9 и проведите для неё **полноценный** эксперимент по шаблону из лекции:

1. Плохой и хороший алгоритм.
2. Независимый эталон (`mpmath` / `decimal`).
3. Диапазон входных значений (минимум 8 точек, логарифмическая сетка).
4. Таблица: вход → результат плохой → результат хороший → эталон → ошибка плохого → ошибка хорошего.
5. Вывод: где и почему алгоритм неустойчив, при каких входных данных он ещё пригоден, какова достижимая точность.

Оформление — как мини-отчёт: цель, постановка, эталон, результаты, вывод. Это финальная задача, показывающая, что вы умеете ставить численный эксперимент **правильно**.

Бонус (по желанию)

Задача 16. Найди неустойчивость в чужом коде.

Найдите в открытом репозитории (GitHub, Stack Overflow) фрагмент кода на Python, где есть потенциальная неустойчивость: вычитание близких чисел, деление на малое, наивная сумма большого массива или ручная реализация `log(1+x)` / `sqrt(x+1)-sqrt(x)`. Приведите код, объясните проблему, предложите устойчивую замену, покажите численно (до/после), что замена помогает. Ссылку на источник — обязательно.

Главное, что проверяется: умение различать обусловленность задачи и устойчивость алгоритма, находить неустойчивость в формулах, ставить эксперимент с независимым эталоном и диапазоном, и делать выводы, а не просто «запустить и посмотреть».